Generar video en pyhton

```
from moviepy.editor import ImageSequenceClip
import numpy as np

# Función para generar imágenes con IA (reemplaza esta función con tu propio código de generación de imágenes)
def generate_images(num_images, image_shape):
    images = []
    for i in range(num_images):
        # Aquí iría tu código para generar una imagen con IA
        # Por ahora, simplemente estamos generando imágenes aleatorias
        img = np.random.randint(0, 256, size=image_shape, dtype=np.uint8)
        images.append(img)
    return images

# Parámetros
num_frames = 100  # Número de frames del video
image_shape = (720, 1280, 3)  # Formato de las imágenes (alto, ancho, canales)
fps = 30  # Fotogramas por segundo del video
output_file = 'generated_video.mp4'  # Nombre del archivo de salida

# Generar imágenes con IA
images = generate_images(num_frames, image_shape)

# Crear un clip de video a partir de las imágenes generadas
clip = ImageSequenceClip(images, fps=fps)

# Guardar el clip de video
clip.write_videofile(output_file, codec='libx264')
```

GENERAR IMAGENES Y VIDEOS CON HILOS EN PYTHON

```
from moviepy.editor import ImageSequenceClip
import numpy as np
import threading
import time

# Función para generar imágenes con IA (reemplaza esta función con tu propio código de generación de imágenes)
#def generate_images(num_images, image_shape):
#    images = []
#    for i in range(num_images):
#        # Aquí iría tu código para generar una imagen con IA
#        # Por ahora, simplemente estamos generando imágenes aleatorias
#        img = np.random.randint(0, 256, size=image_shape, dtype=np.uint8)
#        images.append(img)
#    return images

# Función para generar imágenes con IA (reemplaza esta función con tu propio código de generación de imágenes)
def generate_images(num_images, image_shape):
    images = []
    for i in range(num_images):
        # Aquí iría tu código para generar una imagen con IA
        # Por ahora, simplemente estamos generando imágenes aleatorias
        img = np.random.randint(0, 256, size=image_shape, dtype=np.uint8)
        images.append(img)
    return images

# Parámetros
num_frames = 100  # Número de frames del video
num_threads = 100
threads = []

image_shape = (720, 1280, 3)  # Formato de las imágenes (alto, ancho, canales)
fps = 30  # Fotogramas por segundo del video
output_file = 'generated_video2.mp4'  # Nombre del archivo de salida

# Generar imágenes con IA

#images = generate_images(num_frames, image_shape)

# Crear varios hilos
for i in range(num_threads):
  thread = threading.Thread(target=generate_images(1,image_shape), args=(i,))
  threads.append(thread)
  thread.start()

# Esperar a que todos los hilos terminen su ejecución
for thread in threads:
  thread.join()
```

```
# Crear un clip de video a partir de las imágenes generadas
clip = ImageSequenceClip(images, fps=fps)

# Guardar el clip de video
clip.write_videofile(output_file, codec='libx264')
```

GENERAR VIDEOS CON OPEN CV

```
import cv2
import numpy as np

# Definir las dimensiones del video y la cantidad de cuadros por segundo (FPS)
width = 640
height = 480
fps = 30

# Crear un objeto VideoWriter
video_writer = cv2.VideoWriter('video_creado.avi', cv2.VideoWriter_fourcc(*'XVID'),
fps, (width, height))

# Duración total del video en segundos
duration_seconds = 10

# Generar cada cuadro del video
for frame_count in range(fps * duration_seconds):
  # Crear un cuadro en blanco y negro
  frame = np.zeros((height, width, 3), dtype=np.uint8)

  # Alternar entre blanco y negro en cada cuadro
  if frame_count % 2 == 0:
    frame.fill(255)  # Blanco
  else:
    frame.fill(0)  # Negro

  # Escribir el cuadro en el video
  video_writer.write(frame)

# Liberar recursos
video_writer.release()
```

CREAR UNA IMAGEN ESTILO VAN GOGH

```
import cv2
import numpy as np

# Cargar la imagen de entrada (puedes cambiar la ruta por otra)
input_image_path = "vg3.jpg"
input_image = cv2.imread(input_image_path)

# Cargar una imagen de estilo de Van Gogh (puedes cambiar la ruta por otra)
style_image_path = "balsero.jpg"
style_image = cv2.imread(style_image_path)

# Redimensionar la imagen de estilo para que coincida con la imagen de entrada
style_image = cv2.resize(style_image, (input_image.shape[1], input_image.shape[0]))

# Convertir las imágenes a escala de grises
input_gray = cv2.cvtColor(input_image, cv2.COLOR_BGR2GRAY)
style_gray = cv2.cvtColor(style_image, cv2.COLOR_BGR2GRAY)

# Calcular el flujo óptico entre las imágenes
flow = cv2.calcOpticalFlowFarneback(input_gray, style_gray, None, 0.5, 3, 15, 3, 5,
1.2, 0)

# Aplicar el flujo óptico para crear la imagen de salida
output_image = np.zeros_like(input_image)
for y in range(input_image.shape[0]):
  for x in range(input_image.shape[1]):
    dx, dy = flow[y, x]
    x_new = int(x + dx)
    y_new = int(y + dy)
    if 0 <= x_new < input_image.shape[1] and 0 <= y_new < input_image.shape[0]:
      output_image[y, x] = style_image[y_new, x_new]

# Guardar la imagen resultante
cv2.imwrite("vgf.jpg", output_image)

# Imprimir un mensaje de éxito
print("La imagen con estilo de Van Gogh se ha guardado como
van_gogh_style_output.jpg")
```

Generar Video y añadir Audio

```
import os
import cv2
from moviepy.editor import ImageSequenceClip, concatenate_videoclips
from moviepy.editor import VideoFileClip, AudioFileClip

folder_path = 'imagenes'
output_file = 'generado_video.mp4'
audio = AudioFileClip("musica.mp3")

duracion_imagenes = 2
fps = 30
num_images = 10
image_files = []

def load_images():
    global image_files
    files = [f for f in os.listdir(folder_path) if f.startswith('imagen (') and f.endswith('.jpg')]
    for file in sorted(files)[:num_images]:
        image_path = os.path.join(folder_path, file)
        img = cv2.imread(image_path)
        if img is not None:
            img_resized = cv2.resize(img, (1280, 720))
            image_files.append(img_resized)

load_images()

if image_files:
    image_clips = []
    for img in image_files:
        clip = ImageSequenceClip([img], fps=fps)
        clip = clip.set_duration(duracion_imagenes)
        image_clips.append(clip)

    final_clip = concatenate_videoclips(image_clips)
    final_clip.write_videofile(output_file, codec='libx264')
    print(f"Video generado: {output_file}")
else:
    print("No se encontraron imágenes para generar el video.")

# Establecer el audio del video
video = VideoFileClip("generado_video.mp4")
video = video.set_audio(audio)

# Guardar el nuevo video
video.write_videofile("generado_video.mp4", codec="libx264", audio_codec="aac")
```

Generar Video y añadir Audio 2

```
import os
import cv2
import glob

from moviepy.editor import ImageSequenceClip, concatenate_videoclips
from moviepy.editor import VideoFileClip, AudioFileClip

folder_path = 'imagenes'
output_file = 'generado_video.mp4'
audio = AudioFileClip("musica.mp3")

duracion_imagenes = 2
fps = 30
num_images = 10
image_files = []

def load_images():
    global image_files
    image_files.clear()  # Limpiar la lista si ya tenía datos

    # Obtener archivos JPG
    jpg_files = glob.glob('/content/imagenes/*.jpg')

    if not jpg_files:
        print("⚠️ No se encontraron archivos .jpg en la carpeta.")
        return

    print("🖼️ Archivos encontrados:")
    for file_path in jpg_files:
        print(f"- {file_path}")

        # Leer la imagen con OpenCV
        img = cv2.imread(file_path)
        if img is not None:
            # Redimensionar y guardar en la lista
            img_resized = cv2.resize(img, (700, 720))
            image_files.append(img_resized)
        else:
            print(f"❌ Error al cargar: {file_path}")

load_images()

if image_files:
    image_clips = []
    for img in image_files:
        clip = ImageSequenceClip([img], fps=fps)
        clip = clip.set_duration(duracion_imagenes)
        image_clips.append(clip)
```

```
    final_clip = concatenate_videoclips(image_clips)
    final_clip.write_videofile(output_file, codec='libx264')
    print(f"Video generado: {output_file}")
else:
    print("No se encontraron imágenes para generar el video.")
```

AÑADIR AUDIO

```
!pip install moviepy

from moviepy.editor import VideoFileClip, AudioFileClip
import os

# Rutas
video_path = "/content/generado_video.mp4"
audio_path = "/content/musica.mp3"
output_path = "/content/video_final.mp4"

# Verificar si los archivos existen
if not os.path.exists(video_path):
    raise FileNotFoundError(f"El video {video_path} no existe.")

if not os.path.exists(audio_path):
    raise FileNotFoundError(f"El audio {audio_path} no existe.")

try:
    # Procesar video y audio
    video = VideoFileClip(video_path)
    audio = AudioFileClip(audio_path).set_duration(video.duration)
    video = video.set_audio(audio)

    # Guardar (puede tardar)
    video.write_videofile(output_path, verbose=False, threads=4)
    print("🎵 ¡Audio añadido correctamente!")

except Exception as e:
    print(f"❌ Error: {str(e)}")
finally:
    video.close()  # Liberar recursos
    audio.close()
```